A Quotient Pre-Lie Identity for Proof-Tree Composition Arising from a Primitive Nonmonotonic Consequence Relation

Abstract

We study proof composition generated from a primitive, possibly nonmonotonic, consequence relation. From a base relation $\mid\!\sim_0$ we define a multi-succedent sequent calculus $NMMS(\mid\!\sim_0)$, represent derivations by finite rooted proof trees, and define a grafting operation by insertion at matching leaves. The induced bilinear grafting product on the free abelian group on proof trees admits a refined two-step analysis: every two-step composition is either independent or nested, and the raw witness data may be quotiented by a natural bureaucracy relation. At the quotient level we isolate a quotient-trivial overlap sector (noise), prove that the genuinely mixed residual sector is empty, and show that every residual contribution class is pure outer or pure inner and has a unique partner of the same type on the swapped associator side. The resulting class-level associator is therefore symmetric under interchange of the first two inputs, yielding a class-level pre-Lie identity for proof-tree composition. This provides the proof-theoretic input for the Oudom–Guin construction, and hence for an eventual Hopf-algebraic extension, while keeping the present paper focused on the quotient-invariant structure already established in the formal development.

1 Introduction

The aim of this paper is to extract algebraic structure from proof composition in a setting where monotonicity and familiar structural rules are not built in from the outset. We begin with a primitive consequence relation and use it as the only nonlogical input for a sequent calculus. The resulting

derivations are represented by finite rooted proof trees, and proof composition is defined by grafting one proof tree into another at a leaf with matching sequent interface.

Two distinct issues then arise. The first is combinatorial: how do iterated graftings decompose? The second is proof-theoretic: which apparent distinctions between two-step graftings are genuine and which are merely bureaucratic presentation artefacts? The central contribution of the paper is to show that these two questions can be separated cleanly. The geometry of two-step grafting yields a decomposition into independent and nested cases. Passing to a quotient of raw two-step data isolates the quotient-trivial overlap sector, leaves a rigid residual sector, and makes the associator comparison transparent.

The formal development underlying this paper proves three structural facts that drive the argument.

Theorem 1 (Main structural results)**.** *Let x, y, z, w be proof trees.*

(i) *Every successful two-step grafting from (x, y, z) to w is either independent or nested.*

(ii) *Every mixed residual left contribution class is impossible. Equivalently, every residual left contribution class is pure outer or pure inner.*

(iii) *The class-level associator at output w is symmetric under swapping x and y. Hence proof-tree composition satisfies a class-level pre-Lie identity.*

The key point is that the pre-Lie identity is not obtained by imposing an external tree algebra and then encoding proofs into it. Rather, the proof-theoretic structure itself determines the relevant rooted trees, the grafting operation, the quotient of bureaucratic symmetries, and the resulting class-level associator symmetry.

The presentation is intentionally more conservative than the internal Lean development. We keep only the mathematically significant layers of the construction in the main text:

$$\begin{aligned}\text{primitive consequence} &\longrightarrow \text{derivability}\\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting}\\ &\longrightarrow \text{two-step quotient} \longrightarrow \text{class-level pre-Lie identity.}\end{aligned}$$

The paper is organized accordingly. Section 2 recalls the standard algebraic background on pre-Lie algebras. Section 3 defines the primitive

consequence relation, the calculus $NMMS(\mid\!\sim_0)$, and its induced derivability relation. Sections 4 and 5 develop proof trees, grafting, and the two-step decomposition underlying the associator analysis. Sections 6–8 contain the main quotient argument: bureaucratic equivalence, the separation of noise from residual contribution classes, elimination of mixed residual classes, and the class-level pre-Lie identity. Section 9 records the cut-based coproduct material only as preparatory structure for future work.

2 Algebraic preliminaries

We recall the standard algebraic notion to which the proof-theoretic analysis will ultimately lead.

Definition 2. *A* right pre-Lie algebra *over $\mathbb{Z}$ is an abelian group A equipped with a bilinear product $\triangleright$ satisfying*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z) = (x \triangleright z) \triangleright y - x \triangleright (z \triangleright y)$$

for all $x, y, z \in A$.

Remark 3. *Every right pre-Lie algebra carries an induced Lie bracket*

$$[x, y] := x \triangleright y - y \triangleright x.$$

The present paper does not yet construct the full enveloping Hopf algebra from this bracket. Instead, it establishes the proof-theoretic pre-Lie identity that supplies the input for such a construction.

Remark 4. *Our formal development proves the pre-Lie identity in two complementary forms. First, there is a raw bilinear grafting product on the free abelian group on proof trees. Second, and more importantly for proof identity, there is a class-level pre-Lie identity obtained after quotienting two-step composition data by bureaucratic equivalence. The present paper is primarily about the second form.*

3 Primitive consequence and induced derivability

3.1 Language

Let L_0 be a set of atomic sentence letters. We enlarge L_0 to a language L by adjoining the connectives

$$\bot, \qquad \Rightarrow, \qquad \wedge, \qquad \vee, \qquad \neg.$$

Definition 5. *The set L of formulas is generated inductively by*

$$A ::= p \mid \bot \mid (A \Rightarrow A) \mid (A \wedge A) \mid (A \vee A) \mid \neg A, \qquad p \in L_0.$$

Remark 6. *The constant $\bot$ is treated as an incompatibility marker. We do not assume at the outset any general deduction-theoretic principle reducing incompatibility to some simpler logical notion.*

3.2 Primitive consequence

Definition 7. *A* primitive consequence relation *is a binary relation*

$$\mid\!\sim_0 \subseteq \ MSet(L) \times MSet(L),$$

where $MSet(L)$ denotes the finite multisets of formulas of L.

Remark 8. *The atomic fragment L_0 remains conceptually important as the designated nonlogical vocabulary. Formally, however, the Lean development takes the primitive relation to live on the full language L, and all the theorems in this paper are proved in that setting.*

Remark 9. *No structural conditions are imposed on $\mid\!\sim_0$. In particular, it need not be monotone: from $\Gamma \mid\!\sim_0 \Theta$ one cannot in general infer either $(\Gamma, A) \mid\!\sim_0 \Theta$ or $\Gamma \mid\!\sim_0 (\Theta, A)$.*

3.3 Sequents

Definition 10. *A* multi-succedent sequent *is an expression*

$$\Gamma \mid\!\sim \Delta,$$

where $\Gamma, \Delta \in MSet(L)$.

We work with multisets in order not to presuppose structural features beyond those explicitly built into the proof system.

3.4 The calculus $NMMS(\mid\!\sim_0)$

We now define the nonmonotonic multi-succedent sequent calculus generated by $\mid\!\sim_0$.

Definition 11. *The derivations of $NMMS(\mid\!\sim_0)$ are generated by the following clauses.*

Base axiom. Whenever $\Gamma \mid\!\sim_0 \Theta$, we may form the sequent

$$\frac{\Gamma \mid\!\sim_0 \Theta}{\Gamma \mid\!\sim \Theta} \quad \textit{(Base)}.$$

Implication rules.

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \Rightarrow B), \Gamma \mid\!\sim \Theta} \quad (\Rightarrow L)$$

$$\frac{A, \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \Rightarrow B), \Theta} \quad (\Rightarrow R)$$

Conjunction rules.

$$\frac{A, B, \Gamma \mid\!\sim \Theta}{(A \wedge B), \Gamma \mid\!\sim \Theta} \quad (\wedge L)$$

$$\frac{\Gamma \mid\!\sim A, \Theta \qquad \Gamma \mid\!\sim B, \Theta}{\Gamma \mid\!\sim (A \wedge B), \Theta} \quad (\wedge R)$$

Disjunction rules.

$$\frac{A, \Gamma \mid\!\sim \Theta \qquad B, \Gamma \mid\!\sim \Theta}{(A \vee B), \Gamma \mid\!\sim \Theta} \quad (\vee L)$$

$$\frac{\Gamma \mid\!\sim A, B, \Theta}{\Gamma \mid\!\sim (A \vee B), \Theta} \quad (\vee R)$$

Negation rules.

$$\frac{\Gamma \mid\!\sim A, \Theta}{\neg A, \Gamma \mid\!\sim \Theta} \quad (\neg L) \qquad \frac{A, \Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \neg A, \Theta} \quad (\neg R)$$

Remark 12. *The logical rules are standard active-formula rules. The important structural choice is not the shape of these rules but the absence of built-in structural principles such as weakening. This is the proof-theoretic setting in which the primitive relation $\mid\!\sim_0$ genuinely matters.*

Remark 13. *No weakening or contraction rules are assumed. In particular, the rules*

$$\frac{\Gamma \mid\!\sim \Theta}{\Gamma, A \mid\!\sim \Theta} \qquad \frac{\Gamma \mid\!\sim \Theta}{\Gamma \mid\!\sim \Theta, A}$$

are not built into the system.

Remark 14. *The base relation is extended by logic rather than replaced by logic. In other words, NMMS($\mid\!\sim_0$) is the deductive closure generated from the primitive inferential relation $\mid\!\sim_0$.*

3.5 Induced derivability

Definition 15. *For $\Gamma, \Delta \in MSet(L)$, we write*

$$\Gamma \vdash_{NMMS(\mid\!\sim_0)} \Delta$$

if there exists a derivation of the sequent $\Gamma \mid\!\sim \Delta$ in the calculus $NMMS(\mid\!\sim_0)$.

Remark 16. *This explicitly separates the primitive relation $\mid\!\sim_0$ from the induced derivability relation $\vdash_{NMMS(\mid\!\sim_0)}$. In a nonmonotonic setting, that distinction is mathematically significant.*

4 Proof trees

4.1 Tree representation

Definition 17. *A* proof tree *is a finite rooted ordered tree whose vertices are labelled by sequents, whose internal vertices are tagged by one of the rule symbols*

$$\mathsf{Base},\ \Rightarrow L,\ \Rightarrow R,\ \wedge L,\ \wedge R,\ \vee L,\ \vee R,\ \neg L,\ \neg R,$$

and whose immediate subtrees record the premises of the inference at that vertex. The sequent at the root is called the conclusion *of the proof tree.*

Remark 18. *The order on immediate subtrees is part of the formal object. This is the only reason addresses are later coded by finite sequences of child indices. Conceptually, the objects are ordinary rooted proof trees.*

Proposition 19. *Every derivation d in $NMMS(\mid\!\sim_0)$ determines a proof tree $Tree(d)$ with the same conclusion sequent.*

Proof. The proof is by induction on the derivation:

- a base axiom becomes a leaf;
- each logical rule becomes a node whose children are the proof trees of the premises;
- the root sequent is the conclusion of the inference.

□

4.2 Addresses

Definition 20. *An* address *in a proof tree is a finite sequence of natural numbers indicating a path from the root by successive child indices. The empty address selects the whole tree.*

Remark 21. *Addresses are not conceptually basic. They are simply a convenient way of coding vertices and rooted subtrees in the formal development.*

5 Matching-leaf grafting

5.1 Local grafting

Definition 22. *Let u and t be proof trees. A leaf $a \in Addr(t)$ is* graftable for u *if the sequent labelling that leaf is exactly the conclusion of u.*

Definition 23. *A* matching-leaf grafting *of u into t is the proof tree obtained by replacing a graftable leaf of t by the tree u.*

Definition 24. *We write $Graft(u, t)$ for the finite list of proof trees obtained by matching-leaf grafting of u into t.*

Remark 25. *This operation is controlled by exact interface matching. It does not rely on extending contexts by structural rules.*

5.2 Closure under derivability

Proposition 26 (Closure under derivability)**.** *If $u = Tree(d_{\text{in}})$ and $t = Tree(d_{\text{out}})$ arise from derivations and a is a graftable leaf of t for u, then the grafted tree is again the proof tree of a derivation in $NMMS(\mid\!\sim_0)$.*

Proof. The proof is by induction on the outer derivation. Replacing a premise leaf by another derivation with the same conclusion preserves the validity of the last rule, and hence of the whole derivation. □

Corollary 27. *If u and t are proof trees arising from derivations, then every element of $Graft(u, t)$ also arises from a derivation.*

Proof. Apply the previous proposition to each graftable leaf. □

6 The raw grafting product and two-step decomposition

Let $\mathbb{Z}\langle \mathsf{PT} \rangle$ be the free abelian group on proof trees.

Definition 28. *For proof trees u, t, define*

$$u \triangleright t := \sum_{w \in \mathit{Graft}(u,t)} w$$

and extend this bilinearly to $\mathbb{Z}\langle \mathsf{PT} \rangle$.

Remark 29. *This is the proof-tree analogue of Grossman–Larson grafting: one inserts the rooted proof tree u at every matching leaf of t, but only when the sequent interface matches exactly. The operation therefore depends on proof structure, not merely on the underlying combinatorics of rooted trees.*

Proposition 30. *The coefficient of a proof tree w in $u \triangleright t$ is the number of matching-leaf graftings of u into t yielding w.*

Remark 31. *The raw product already satisfies the pre-Lie identity in the formal development. The point of the quotient analysis below is not to rescue a false algebraic claim, but to explain that identity in proof-theoretic terms by isolating the bureaucratic redundancy in two-step composition.*

6.1 Two-step graftings

Fix proof trees x, y, z, w. A *two-step grafting from (x, y, z) to w* means first grafting y into z at a matching leaf to obtain an intermediate tree z', and then grafting x into z' to obtain w.

Definition 32. *Such a two-step grafting is called* outer *if the second graft occurs independently of the first, and* inner *if the second graft occurs inside the subtree introduced by the first. In address terms, these are the incomparable and comparable cases.*

Theorem 33 (Two-step decomposition)**.** *Every successful two-step grafting is either nested or independent. More precisely, if*

$$y \text{ grafts into } z \text{ to yield } z' \qquad \text{and} \qquad x \text{ grafts into } z' \text{ to yield } w,$$

then exactly one of the following geometric descriptions applies:

(i) Inner case: *the second address extends the first, so the second graft occurs inside the subtree inserted by the first;*

(ii) Outer case: *the two addresses are incomparable, and the two insertions can be performed in the opposite order.*

Proof sketch. This is the content of the full address analysis formalized in the Lean theorem 'two_step_graft_decomposition_full'. If the second address begins with the first, one is in the nested case. Otherwise the addresses are incomparable, and the incomparable-address commutation theorem shows that the order of insertion can be reversed. □

Remark 34. *This theorem is the geometric heart of the associator analysis. It shows that the only essential distinction among two-step compositions is between independent and nested insertion.*

Theorem 35 (Commutation of independent graftings)**.** *If two successful graftings occur at incomparable addresses, then performing them in either order yields the same final proof tree.*

Proof sketch. Incomparable addresses refer to disjoint subtrees. Replacing the subtree at one address does not affect the subtree at the other, so the corresponding local modifications commute. □

7 Quotienting bureaucratic two-step data

At raw address level, two-step configurations still contain too much presentation data. The next step is to isolate their genuine dependency geometry.

7.1 Two-step classes

Definition 36. *A* two-step code *records a successful two-step grafting together with its geometric type: left outer, right outer, left inner, or right inner. Here "left" and "right" refer to the two bracketings*

$$x \triangleright (y \triangleright z) \qquad \textit{and} \qquad y \triangleright (x \triangleright z),$$

while "outer" and "inner" refer to the independent and nested cases of the two-step decomposition.

Definition 37. *The* bureaucratic equivalence *on two-step codes is the least equivalence relation generated by:*

(i) *commutation of independent outer insertions;*

(ii) *transport of outer left presentations to the corresponding swapped outer presentations;*

(iii) *transport of nested left presentations to the corresponding swapped inner presentations;*

(iv) *the inverse transports from the swapped side.*

Definition 38. *The quotient set of two-step codes is denoted*

$$\mathcal{Q}(x, y, z; w).$$

Inside it we distinguish the left contribution classes

$$\mathcal{C}_{\mathrm{L}}(x, y, z; w)$$

and the swapped-right contribution classes

$$\mathcal{C}_{\mathrm{R}}(x, y, z; w),$$

the latter being associated with the swapped bracketing (y, x, z).

Remark 39. *The quotient is meant to identify proof presentations that differ only by bureaucratic scheduling or by equivalent ways of viewing the same nested composition. This is the formal counterpart of the proof-theoretic idea that identity of proof should ignore inessential order of independent operations.*

7.2 Canonical transport theorems

Theorem 40 (Outer transport)**.** *For every* x, y, z, w*, outer left contribution classes are canonically in bijection with outer contribution classes on the swapped right side.*

Proof sketch. If the two insertions are independent, they occur at incomparable addresses. The incomparable-address commutation theorem reverses their order without changing the final proof tree. This witness-level transport descends to the two-step quotient and yields a bijection of outer classes. □

Theorem 41 (Inner transport)**.** *For every* x, y, z, w*, inner left contribution classes are canonically in bijection with inner contribution classes on the swapped right side.*

Proof sketch. In the nested case, one uses the inner branch of the two-step decomposition: instead of viewing the second graft as occurring inside the previously inserted subtree, one first performs the inner graft inside the inserted tree and then reinserts the result into the ambient proof tree. Again, the transport respects the two-step quotient. □

Remark 42. *The outer transport is a strict commutation theorem. The inner transport is a reassociation theorem. Their common role is to show that the quotient keeps the geometric content of two-step composition while discarding the accidental presentation details.*

8 Noise, residual classes, and purity

8.1 Noise and residual sectors

Definition 43. *A left contribution class is called* noise *if the apparent difference between its outer and inner presentations is already quotient-trivial under the comparison with the swapped side. A left contribution class is* residual *if it is not noise.*

Remark 44. *"Noise" does not mean mathematically uninteresting in itself. It means precisely that the class contributes no genuine asymmetry to the associator comparison, because the quotient already identifies the competing presentations.*

Definition 45. *A residual left contribution class is called:*

- pure outer *if it has outer support and no inner support;*
- pure inner *if it has inner support and no outer support;*
- mixed *if it has both outer and inner support.*

Theorem 46 (Elimination of mixed residual classes)**.** *There are no mixed residual left contribution classes.*

Proof sketch. Suppose a left contribution class carried both outer and inner support while remaining residual. The geometric analysis of open mixed classes first shows that every outer representative of such a class must lie in a dependent overlap regime. The crucial step is then that a comparable outer witness automatically produces opposite-side inner support. But opposite-side inner support is one of the quotient-trivial overlap mechanisms already isolated by the definition of noise. Thus the class must in fact be noise, contrary to assumption. □

Corollary 47 (Residual purity)**.** *Every residual left contribution class is either pure outer or pure inner.*

Proof. By the theorem, the mixed case is impossible. □

Remark 48. *This is the decisive structural step in the whole proof. Once mixed residual classes disappear, the residual sector becomes typed and rigid, and the final pre-Lie comparison reduces to separate outer and inner transport theorems.*

8.2 Unique typed partners

Theorem 49 (Unique typed partner theorem)**.** *Let* $q \in \mathcal{C}_{\mathrm{L}}(x, y, z; w)$ *be residual.*

(i) *If* q *is pure outer, then there exists a unique pure outer class* $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ *corresponding to* q*.*

(ii) *If* q *is pure inner, then there exists a unique pure inner class* $q' \in \mathcal{C}_{\mathrm{R}}(x, y, z; w)$ *corresponding to* q*.*

Proof. If q is pure outer, apply the outer transport theorem. If q is pure inner, apply the inner transport theorem. Residual purity guarantees that these cases are disjoint, and the transport maps are equivalences on the relevant typed sectors. □

Theorem 50 (Residual classes seen from the swapped side)**.** *Every residual swapped-right contribution class arises from a pure residual left contribution class.*

Proof sketch. The class-level decomposition on the swapped side yields either a noise class or a residual class. In the residual case, one transports back to the left by the class-level comparison relation and then uses residual purity on the left. □

9 The class-level pre-Lie identity

9.1 Associator contributions at a fixed output

Definition 51. *For proof trees* x, y*, define the* primitive class-level product support *by*

$$\mathsf{Prod}(x, y) := \{\, w \in \mathsf{PT} : w \in \mathit{Graft}(x, y) \,\}.$$

Remark 52. *This is not yet a quotient object. It is simply the set of outputs occurring in one-step grafting. The quotient enters only in the analysis of how these outputs receive two-step associator contributions.*

Definition 53. *Fix x, y, z, w. The* class-level associator at output w *is the assertion that left contribution classes to w decompose into noise, pure outer residual classes, and pure inner residual classes, and that these three sectors compare with the swapped side exactly as described by the noise, outer-transport, and inner-transport theorems.*

Theorem 54 (Class-level associator shape)**.** *For every x, y, z, w, the class-level associator at output w holds.*

Proof sketch. Every left contribution class is either noise or residual. By residual purity, every residual class is pure outer or pure inner. Noise is irrelevant to the associator comparison by definition; pure outer residual classes have unique outer partners by the outer transport theorem; pure inner residual classes have unique inner partners by the inner transport theorem. Finally, every non-noise swapped-right class comes from a pure residual left class. □

Theorem 55 (Class-level pre-Lie comparison)**.** *Fix x, y, z, w. Every left contribution class to the coefficient of w in the associator belongs to exactly one of the following three sectors:*

(i) *quotient-trivial noise;*

(ii) *pure outer residual, with a unique swapped pure outer partner;*

(iii) *pure inner residual, with a unique swapped pure inner partner.*

Proof. This is the operational form of the class-level associator shape. □

9.2 The main theorem

Theorem 56 (Class-level pre-Lie identity)**.** *For all proof trees x, y, z, the class-level associator is symmetric under interchange of x and y. Equivalently, for every output tree w, the contribution classes to*

$$(x \triangleright y) \triangleright z - x \triangleright (y \triangleright z)$$

are matched, after quotienting by bureaucratic equivalence, with the contribution classes to

$$(y \triangleright x) \triangleright z - y \triangleright (x \triangleright z).$$

Proof. Fix w. By the class-level pre-Lie comparison theorem, every left contribution class lies in exactly one of three sectors.

Noise. Noise classes are quotient-trivial by construction and contribute no genuine asymmetry.

Pure outer residual classes. These are matched uniquely across the two bracketings by outer transport.

Pure inner residual classes. These are matched uniquely across the two bracketings by inner transport.

Since these three sectors exhaust the left contribution classes at output w, the class-level associator at w is symmetric under swapping x and y. As w was arbitrary, the class-level pre-Lie identity follows. □

Remark 57. *This theorem is the precise endpoint currently established by the formal development. It is a statement about quotient-invariant associator comparison at the level of contribution classes. It does not yet claim a fully elaborated Hopf algebra in the main body of the paper.*

10 Preparatory coproduct data and Hopf outlook

The formal development also defines a cut-based coproduct datum on proof trees. We record only the part that is presently used with mathematical confidence in the current project.

Definition 58. *An* admissible cut *of a proof tree is a finite antichain of addresses whose associated cut decomposition respects the base-interface constraints tracked in the formal development. The cut determines a pair consisting of a forest of extracted subtrees and a remainder tree.*

Proposition 59. *Every proof tree admits at least one cut decomposition.*

Proof. The empty cut is always admissible, and it yields the trivial decomposition into the empty forest together with the original tree. □

Remark 60. *This coproduct material is preparatory rather than central in the present paper. It shows that the proof-tree side already carries the decomposition data expected of a Connes–Kreimer style coproduct, but we do not yet prove the full compatibility theorems needed for a Hopf algebra in the body of the paper.*

Remark 61. *There are two natural routes from the present work to Hopf algebra.*

(i) *A direct cut/grafting route, in which one relates the cut-based coproduct to proof-tree grafting itself.*

(ii) *An Oudom–Guin route, in which the class-level pre-Lie structure proved here is first extended to a symmetric algebra and only then combined with a primitive coproduct.*

The second route is the one most strongly suggested by the present formal results, and it is the route we have in mind when the abstract mentions Hopf algebraic consequences.

Remark 62. *A further comparison problem, not treated here, is to relate the present structure to familiar rooted-tree Hopf algebras by forgetting some of the proof-theoretic decoration while retaining the underlying tree and insertion geometry. Such comparison theorems would strengthen the claim that the present construction refines, rather than competes with, established combinatorial Hopf-algebraic frameworks.*

11 Conclusion

We have constructed a proof-theoretic route from a primitive consequence relation to a class-level pre-Lie identity on proof-tree composition. Formally, the route is

$$\begin{aligned} \mid\!\sim_0 &\longrightarrow NMMS(\mid\!\sim_0) \\ &\longrightarrow \text{proof trees} \longrightarrow \text{grafting geometry} \\ &\longrightarrow \text{quotient of two-step composition} \longrightarrow \text{class-level pre-Lie identity.} \end{aligned}$$

The distinctive point is not merely that rooted trees appear, but that they appear as proof objects carrying exact sequent interfaces. Grafting is therefore proof composition, not arbitrary combinatorial insertion. The quotient on two-step data then isolates what should count as invariant in proof composition: independent commutations and quotient-trivial overlap disappear as bureaucracy, while the residual sector becomes pure and rigid enough to satisfy the class-level pre-Lie identity.

From the standpoint of logic, this means that a nonmonotonic proof theory can carry nontrivial algebraic structure without first collapsing to a monotone or structurally saturated setting. From the standpoint of algebra, it shows that pre-Lie behavior can emerge from proof composition once identity of proof is handled carefully at the level of two-step interactions. The Hopf-theoretic extensions suggested by the present development are

important, but they are best seen as a next step built on top of the logic result already proved here: after quotienting bureaucratic symmetries, proof composition on these nonmonotonic proof trees satisfies a class-level pre-Lie identity.